

Hardware-Praktikum 2026

O. Amft und L. Häusler, Intelligent Embedded Systems Lab,
Institute for Computer Science, Albert-Ludwigs-University Freiburg, Freiburg im Breisgau

Exercise sheet 2 (26 Punkte)

Goal:

The goal is to be able to play monophonic ringtones in the *Ring Tone Text Transfer Language* (RTTTL) in the background. This will be achieved using hardware related instructions, mainly Interrupt Service Routine (ISR). You can use the template provided for better structuring. There are several hints at the end of this sheet to make it searching easier but if you want to challenge yourself, all needed information can be found in documentations available.

Preparation:

In the days of MicroPython¹, eLua², and Espruino³, it's easy to forget that microcontrollers have limited resources. For example, the memory of the XIAO nRF52840 (256KB) is just enough to store the following image 8 times (32 KB 200x200 pixels as PNG).

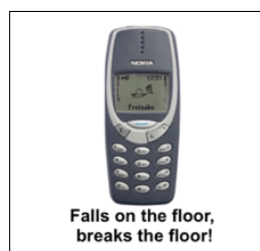


Abbildung 1: Nokia3310 (year 2000) based on an ARM7 32 bit processor with 13MHz.

The first cell phones developed in the late 1990s also made do with limited resources. For example, the Nokia3310 shown in the picture contained a single-core processor clocked at 13MHz. It could make phone calls, write SMS and play simple games like “Snake”.

Ringtones were stored in RTTTL format by Nokia (and other manufacturers) and allowed to play simple monophonic melodies, which you could also create yourself.

You now have the task to take over a part of the development of the Nokia team from the year 1999. You have to develop a RTTTL player that can play a ringtone in the background - you want to be able to navigate through the menu while the ringtone is playing. Ringtones consist of a sequence of musical notes. A note consists of pitch and duration. There can also be pauses of appropriate length between notes. Only one note can be played at any time (hence monophonic). To play a note, sound waves with the appropriate carrier frequency must be generated. In cell phones of that time and also on our extension board, a Buzzer is built in for this purpose. The frequency of musical notes can be found online.

The tasks are structured in such a way that you gradually implement the functions required for the overall task. Since the Arduino project, among others, did not yet exist in 1999, it was necessary to program close to the hardware - in some cases probably even in assembler. However, we limit ourselves in this sheet to a hardware-near programming in C.

¹ <https://micropython.org> ² <http://www.eluaproject.net> ³ <http://www.espruino.com>

Notes:

Refer the code examples on ILIAS, which contain many tips and examples. Also note that your code should be adequately commented. This is especially true when writing or reading registers. You also need the datasheet of the nRF52840 Sense (Small tip: shortcut to search function is usually Ctrl+F). Everything about the chip is explained there. There are further hints for each exercise on the last pages of this PDF.

Task 1 (2 points):

To understand how to set a pin to 0 or 1 without using the `digitalWrite()` function, you are required to implement a function called `setP026(boolean high)`.

Refer to the examples provided on Ilias (or at the end of this PDF) for guidance. Please note that since the architecture of the ATmega328 (used in the examples) differs from the nRF52840, you will need to adapt the code to write to the appropriate registers for our hardware.

Use the created function and toggle the pin with a frequency of 1 Hz. The Red LED on the nRF52840 should light up accordingly.

Task 2 (4 points):

To play the note C_6 with the Buzzer, it must generate sound waves at 1046 Hz. To do this, a frequency of 1046 Hz must be present at the corresponding microcontroller pin. To generate such a frequency, first (if you haven't already) connect the nRF52840 to the extension board. Now design a function `setTimer1Freq` which, using timer 1 and an interrupt, generates a square wave signal with a frequency of 1046 Hz (1046 Hz are audible) and plays C_6 on the buzzer. Make sure you have the right frequency by testing the generated sound.

Task 3 (1 points):

Now that the timer and interrupt systems are functional, your task is to implement a controlled frequency output. write the function `setBuzzerFreq`, based on `setTimer1Freq`. In addition to the name change, you must implement a trigger mechanism to start and stop the output. You can use the button on extension board.(if you choose to use the onboard microphone as a trigger, ensure you implement appropriate thresholds to prevent accidental activation from ambient noise)

Task 4 (4 points):

To be able to play other tones besides $C_6 = 1046$ Hz, extend `setBuzzerFreq` with a parameter for the desired frequency, which should be able to be between 100 Hz and 3 kHz. The function should change the *Compare Register* and possibly the *Prescaler*, but not change the entire timer. For frequencies outside the above range, the pin should be set to *low* and the timer disabled.

Task 5 (5 points):

Unfortunately, an active delay prevents the controller from doing other tasks in the time. To make this concurrent and only reconfigure the timer after the delay has expired, we can use interrupts. Write a function `setTimer2` that configures timer 2 to fire an interrupt every 1 ms. Also, implement the corresponding ISR. Through a parameter, `setTimer2` should be able to enable or disable the timer. Also introduce a new global variable `volatile uint32_t tCount`⁴, which is incremented on each interrupt (in the ISR). Test your implementation by periodically (e.g. 1x per second) outputting the current value of `tCount` via the serial interface in the *loop*.

Initialize the serial interface once in the *setup* with 115200 baud.

⁴ The keyword *volatile* is needed to tell the compiler that the value of `tCount` can also change outside the “normal” program flow

Task 6 (4 points):

Introduce two new global arrays `uint16_t durations[10]` and `uint16_t notes[10]`. Here the value `durations[0]` corresponds to the duration (in milliseconds) and `notes[0]` corresponds to the frequency (in Hz) of the first note of the melody to be played. Fill both arrays with 10 values between 200 and 1000. If you are familiar with *structs*, you can define an appropriate type (e.g. *struct Note*) instead.

Also introduce the new global variable `volatile uint8_t melodyIdx` (initialized to 0). `durations[melodyIdx]` and `notes[melodyIdx]` should correspond to the note and duration of the melody currently being played.

Extend the ISR from task 5 to include the following functionality:

When $tCount \geq durations[melodyIdx]$, `tCount` is set back to 0, `melodyIdx` is incremented, and the next frequency of the melody to be played is created (using `notes` and `setBuzzerFreq`). Note that `melodyIdx` may have a maximum value of 9. If the value reaches 10, the timer should be deactivated (via `setTimer2`). Implement another function `playMelody` that sets the appropriate initial values for the `index`, and calls `setBuzzerFreq` already with the first sound to be played. In addition, this function should call the `setTimer2` implemented in Task 5 accordingly. Your implementation should now be able to play a short melody once when you execute `playMelody` at the end of `setup`.

Task 7 (6 points)

Now we have all the required functions together to play ringtones. However, we still need to parse them from the appropriate format and store them in the corresponding arrays `durations` and `notes`. Monophonic ringtones are encoded in the Ring Tone Text Transfer Language (RTTTL)⁵. Consequently, the ringtone serves as an example for the explanation of RTTTL:

Test:d=4,o=5,b=200:8g,8a,8c6,8a,e6,8p,e6,8p,d6.,8p,8g,8a,8c6,8a,d6,8p,d6,8p,c6,8p,a.,8g,8a,8c6,8a,2c6,d6,b,a,g.,8p,g,2d6,2c6.,p,8g,8a,8c6,8a,e6,8p,e6,8p,d6.,8p,8g,8a,8c6,8a,g6,b,c6,8p,b,8a,p,8g,8a,8c6,8a,2c6,d6,b,a,g,p,g,d6,c6

Each RTTTL ringtone starts with a name (which can be empty, in the example: “Test”) followed by a “:”.

After that default parameters for this ringtone are set:

- *d* stands for *duration* and describes the default note value (whole, half, quarter, ...). $d = 4$ in the example indicates that all notes for which it is not explicitly stated otherwise are quarter notes. The possible values are: $d \in [1, 2, 4, 8, 16, 32]$.
- *o* stands for *octave* and describes the default octave. $o = 5$ in this example indicates that all notes for which it is not explicitly stated otherwise are to be selected from the 5th octave. The possible values are: $o \in [4, 5, 6, 7]$.
- *b* stands for *beats per minute* and describes the default duration of a quarter note. $b = 125$ in this example specifies that all **quarter** notes should have a duration of $60 \cdot 1000 / 125 = 480 \text{ ms}$. The possible values are: $b \in [25, \dots, 900]$.

The order of the parameters is fixed (first *d*, then *o*, and finally *b*), but the parameters are each optional. The default values are $d = 4$, $o = 6$, $b = 63$. The parameters are again separated from the actual notes by “:”.

Individual notes are themselves separated with “,”. Each note can consist of the following components: `<duration><note><octave><dot>`. Everything except `<note>` is optional in this case.

- `<note>` describes the note to play ($\in [c, c\#, d, d\#, e, f, f\#, g, g\#, a, a\#, b, p]$). *b* may also be encoded as *h* in some RTTTL melodies, but they don't necessarily have to catch this case. *p* here is a special case for a rest note (frequency = 0 Hz).

⁵ for more information on RTTTL, see here: http://www.mobilefish.com/tutorials/rtttl/rtttl_quickguide_specification.htm

- `<duration>` is identical to the `d` parameter. If `<duration>` is empty, the default parameter applies. `<octave>` is identical to the `o` parameter. If `<octave>` is empty, the default parameter applies.
- If a `<dot>` = “.” is appended, the sound is extended by half its duration. Example: “4g.” is a quarter note `g` extended by an eighth note `g`.

To test your implementation, first store the *Test* ringtone into a global array `char buffer[] = “test:...”;`. Parse the individual parts of the string and translate the notes into the appropriate frequencies and lengths and store them accordingly in the arrays `durations` and `notes` (or `struct` if necessary).

caution If applicable, the array size must be adjusted accordingly. Use a maximum size (e.g. 100) and another global variable `melodyLen` which is set to the number of tones to play. After parsing the ringtone, it should be played once in the background. To do this, use the `playMelody` function from task 6.

Bonus: (3 points) play and listen to examples provided in template. Change the name of arrays to corresponding song. Show the name of current song on display of extension board, and make it so that you can change between songs by pressing the button on extension board.

Submission:

Archive all your programs in a ZIP file and upload it to the exercise portal. Please provide one program for each template file. Note that tasks 2 and 3 should be combined into a single program. Code from tasks that are modified or no longer needed should be commented out. Add appropriate comments (Ex: `// Ex02: Configuration of Timer 1 for 1046Hz ...`). Check the archive file for completeness.

Background

Task 1:

To understand how to control a pin without using the Arduino `digitalWrite()` function, you must interact directly with the microcontroller's registers. While the examples provided on Ilias use the ATmega328 (AVR) architecture, our nRF52840 operates differently. You will need to adapt the logic from the examples to fit the nRF52 register map.

1. Bit Operations

Hardware registers are controlled via individual bits. To manipulate a specific bit n without affecting others, use the following patterns:

```
R |= (1UL << n);    // set bit n to 1
R &= ~(1UL << n);   // clear bit n to 0
```

2. Architecture Comparison

- **Arduino (AVR):** Uses registers like `DDRB` (direction) and `PORTB` (state). For example, Pin 13 is bit 5 of `PORTB`.
- **nRF52 (ARM):** Pins are organized in 32-bit ports (`P0`, `P1`, etc.). For example, pin `P0.26` corresponds to bit 26 of port 0 (`NRF_P0`).

3. nRF52 Register Logic

Refer to the datasheet (GPIO → Registers) for the following:

- `DIRSET` / `DIRCLR` → Set the pin direction (output/input).
- `OUTSET` / `OUTCLR` → Set the pin state (HIGH/LOW).

Example Implementation:

```
NRF_P0->DIRSET = (1UL << 26);    // Set P0.26 as output
NRF_P0->OUTSET = (1UL << 26);     // Set P0.26 HIGH
NRF_P0->OUTCLR = (1UL << 26);     // Set P0.26 LOW
```

Important Notes:

- **Initialization:** A pin must be set as an output before it can drive a signal.
- **Atomicity:** `OUTSET` and `OUTCLR` are atomic; they do not require a read-modify-write cycle.
- **Typing:** Always use `1UL << n` to ensure the shift is treated as an unsigned 32-bit integer.
- **Alternative Pins:** You may use the other two available LED pins (green or blue) instead of `P0.26`. If you do, please add a comment in your code specifying which pin was used.

Task 2:

To play a note (like C6 at 1046 Hz), the microcontroller must toggle a pin at a precise frequency. A square wave is generated by switching a pin between HIGH and LOW every half-period ($T_{half} = 1/(2 \cdot f)$).

The code examples provided on Ilias (using `TCCR2A`, `OCR2A`, and `ISR`) are for the AVR (**ATmega328**) architecture. The nRF52840 (ARM Cortex-M4) uses a different timer system based on **Tasks, Events, and Shortcuts**.

nRF52 Timer Guide:

- Use `NRF_TIMER1`. You must define the `MODE` (Timer), `BITMODE` (e.g., 32-bit), and the `PRESCALER` to determine the tick rate.
- The `CC[0]` (Capture/Compare) register determines when an event is triggered. For 1046 Hz, calculate the number of ticks required for T_{half} .

- To create a periodic wave, use a `SHORT` to automatically clear the timer when it reaches the `CC[0]` value.
- Use `INTENSET` to enable the compare interrupt and `NVIC_EnableIRQ(TIMER1_IRQn)` to enable it in the ARM processor.
- The interrupt service routine must be defined as `extern "C" void TIMER1_IRQHandler()`. Inside, you must check and clear the `EVENTS_COMPARE[0]` flag.
- Buzzer pin corresponds to register `NRF_P0`, bit 29.
- You can analyze the frequency with an appropriate smartphone app or simply compare it with an appropriate tone (see for example the Online Tone Generator at <https://www.szynalski.com/tone-generator/> - here you can also set a square wave signal).
- Since you are not generating a perfect sine wave, but a square wave signal, the frequency spectrum also contains harmonics $n \cdot f_0$ of the base frequency ($f_0 = 1046$ Hz). If you use a smartphone for frequency analysis, it is possible that a higher frequency component is detected for one of the harmonics (see figure 2). However, a certain amount should also be present in the fundamental (see cursor in Figure 2).



Abbildung 2: Spectrum of a 1046 Hz Tons generated by a buzzer. (iOS App “Spectrum Analyzer”)

Task 3:

- Button on extension board corresponds to register `NRF_P0`, bit 3.

Task 7:

You are given every freedom in the implementation. However, one possible approach is outlined below:

1. First determine the total length of the *buffer* using, for example, the `strlen` function and run over the *buffer* with an *index* variable.
2. You can skip the name by running to the next `','`.
3. Parse the default values first. An example of how this can be done is shown in line 24 of Listing 1.
4. When parsing the individual notes, it is recommended to use a *for* or *while* loop and carry an index for the length of the melody.
 - (a) First parse the `<duration>` (if specified - otherwise the default duration applies).
 - (b) Now parse the `<note>` (single character). For example, you can use a *case* construct to convert the `<note>` to the corresponding frequency of a fixed octave.
 - (c) If the next character is `'#'` you still have to raise it by a semitone. (This step can also be included in the *case* construct).
 - (d) Parse the `<octave>` (if specified - otherwise use the default octave).
 - (e) Change the note frequency depending on the *octave* (see notes).
 - (f) If the next character is `','`, you must increase `<duration>` by half of it.
 - (g) According to the *bpm* you can now calculate the duration of the note in milliseconds.

- (h) Store the values corresponding to the loop index into the *durations* and *notes* array.
- (i) If the next character is not a ',', you have reached the end of the melody.

5. Finally, you can now play the melody by calling *playMelody*.

Notes:

- The frequency ($f(N_{oct})$) of a note (N) has an exponential relationship with the octave (oct): $f(N_{oct}) = f(N_0) \cdot 2^{oct}$. So you only have to store the frequencies of one octave (e.g. the smallest possible one ($o = 4$)) and can calculate the others accordingly.
- For pauses ($\langle note \rangle = p$) you can simply choose a frequency of 0 Hz. You should have already caught the special case in the *setTimer1Freq* function.
- To convert a character array e.g. "16" to an integer 16, you can for example proceed as shown in Listing 1. In the code of *parseRTTTL* starting from line 24 the use of the functions *charIsDigit* and *str2uint* is shown. Here the *standard duration* is read after the name of the ringtone. All other values like the *standard octave* and *bpm*, as well as $\langle duration \rangle$ and $\langle octave \rangle$ of each note can be read analogously.

```

1 // Check if the given character is a valid digit
2 bool charIsDigit(char c) {
3     // '0'-'9' are ascii encoded
4     if (c < '0' || c > '9') return false;
5     return true;
6 }
7 // Convert the digits at the current and following
8 // buffer position idx into an unsigned integer
9 uint16_t str2uint(char * buffer, uint16_t * idx) {
10    uint16_t val = 0;
11    // While character is a valid digit
12    while (charIsDigit(buffer[*idx])) \end{enumerate}
13        // '0' is 48 ascii encoded, '1' is 49. '2' is 50 etc.
14        val = (val * 10) + (buffer[*idx] - '0');
15        // Increment index (* needed to address pointer)
16        *idx = *idx + 1;
17    }
18    return val;
19 }
20
21 void parseRTTTL() {
22     ...
23     // Example of parsing optional standard duration
24     if (buffer[idx] == 'd') {
25         idx += 2; // skip 'd'+'='
26         if (charIsDigit(buffer[idx])) {
27             // That's how you call the above function and pass parameters as pointers
28             standardDuration = str2uint(buffer, &idx);
29         }
30         idx++; // Skip comma
31     }
32     ...
33 }

```

Listing 1: example to convert *characters* to *integers* and parse the *standard duration*